Detección de Fraude Bancario con el modelo KDD

Informe técnico · Proyecto Final SINF-633 Análisis Exploratorio de Datos, Universidad Cenfotec

1. Resumen ejecutivo

En este proyecto tomamos la situación de hacer una implementación de un modelo de detección o prevención de fraude bancario con KDD, desde una perspectiva de analista de datos y aplique las 5 etapas del modelo KDD (Fayyad et al., 1996) sobre 5 000 transacciones del dataset. La pregunta que quería responder es: **¿Qué patrones permiten distinguir transacciones legítimas de fraudulentas, y cómo puede el banco usar esos patrones para reducir pérdidas sin molestar al cliente legítimo?**

Lo principal que encontré:

- Solo el 2.18% de las transacciones son fraude (109 de 5 000), o sea hay aprox. 45 legítimas por cada "unidad" de fraude.
- El fraude se concentra en la combinación de **monto alto + horario de noche** (22:00 a 05:59). En el 20% de montos más altos durante la noche aprox. el 24% de las transacciones son fraude, y en el resto de combinaciones no pasa de 1.5%.
- Varias técnicas distintas llegan a lo mismo: la tabla cruzada, K-Means, Apriori y la importancia de variables del Random Forest.
- El mejor modelo fue el **Random Forest**: con el umbral normal todas sus alertas son fraude real (precision de 100%) y detecta 16 de 22 fraudes. Con un umbral de 0.3 detecta 18 de 22 molestando solo a 1 cliente legítimo de 978.
- Propongo usar el modelo con 3 niveles de riesgo (aprobar, pedir OTP o bloquear), así la gran mayoría de clientes no siente ninguna molestia.

2. Datos y metodología

Etapa 1: Selección

Cargue el archivo *transacciones_bancarias.csv* con Pandas usando *parse_dates* en *fecha_hora*, para que el sistema lea la fecha como fecha y no como texto, y así poder sacar la hora. Las variables las clasifique así:

- **Numéricas:** edad_cliente, antiguedad_cuenta_meses y monto. Se usan (el monto como monto_log).
- **Categóricas:** tipo_transaccion, canal y region. Se usan.
- **estado_transaccion:** no la use en el modelo porque el estado se conoce después de procesar la transacción, y el modelo tiene que decidir antes. Usarla seria aprender con información del futuro (data leakage). Solo la use en Apriori, que explora datos históricos.
- **Fecha:** fecha_hora, la use para crear la hora y saber si fue de noche.
- **Identificadores:** id_transaccion e id_cliente. No se usan porque son códigos, no miden nada y le enseñarían al modelo por ID y no por comportamiento. Además los 109 fraudes están repartidos en 107 clientes distintos.
- **Objetivo:** es_fraude (1 = fraude, 0 = legítima).

Por el desbalance, un modelo que diga siempre "legítima" acertaría el 97.8% sin detectar ningún fraude, por eso no consideré solo en la accuracy y evalué con recall, precision, F1 y AUC.

Etapa 2: Preprocesamiento

- **Nulos:** edad_cliente tenía 71 (1.42%) y los rellene con la mediana porque no se afecta por valores extremos. canal 59 (1.18%) y region 79 (1.58%) los rellene con la categoría "Desconocido" para no inventar un dato que no sabemos.
- **Outliers (IQR, k = 1.5):** monto tiene 584 outliers con 13.87% de fraude contra 0.63% del resto, aprox. 22 veces más. Decidí conservarlos porque eliminarlos seria borrar los casos que queremos detectar. Edad_cliente tuvo 18 outliers (sin fraude) y antiguedad_cuenta_meses ninguno.
- **Duplicados:** no se encontró ninguno, así que no se elimino ninguna fila.
- **Logaritmo:** aplique np.log1p() al monto y la asimetría bajo de 8.66 a 1.64. Nos ayuda a que no hayan muchas distorsiones dentro de la data que vamos a usar.

Etapa 3: Transformación

- Cree 3 variables nuevas: **monto_log**, **hora_dia** (0 a 23) y **es_noche** (1 si es entre 22:00 y 05:59).
- En la matriz de correlación ningún par supera r > 0.8, así que no hay multicolinealidad fuerte. El par más alto es monto con monto_log (0.59) porque una sale de la otra, por eso en el modelo solo use monto_log.
- Dividí los datos 80/20 con stratify=y y semilla 42 (87 fraudes en train y 22 en test).
- Arme un Pipeline con OneHotEncoder(drop_first) para las categóricas y StandardScaler para las numéricas, ajustado solo con train para que el modelo no vea información del test.

Etapa 4: Minería de datos

- **No supervisado** (con 5 variables numéricas escaladas): K-Means con k de 2 a 10, DBSCAN (eps = 0.8 con el gráfico de k-distancias y min_samples = 10, que es 2 × 5 variables) y PCA a 2 y 3 componentes.
- **Supervisado:** En el supervisado usamos regresión Logística y Random Forest, los dos con class_weight="balanced" para darle más peso a los fraudes para que los datos no esten con una varianza muy grande, y Gradient Boosting como modelo extra.
- **Bonus:** reglas de asociación con Apriori.

3. Resultados cuantitativos

Modelos supervisados (test: 1 000 transacciones, 22 fraudes)

Modelo	Accuracy	Precision	Recall	F1	AUC-ROC
Regresión Logística	0.834	0.100	0.818	0.178	0.906
Random Forest	0.994	1.000	0.727	0.842	0.894
Gradient Boosting	0.993	0.895	0.773	0.829	0.889

La Regresión Logística detecta más fraudes (18 de 22), pero de 180 alertas solo 18 son fraude, o sea molestaría a 162 clientes legítimos. El Random Forest detecta 16 de 22 pero todas sus alertas son fraude real. La Regresión Logística tiene el AUC más alto porque ordena bien el riesgo en general, pero con el umbral normal lanza demasiadas alertas. Elegí el Random Forest porque es el que mejor equilibra atrapar fraudes y no molestar, y funciona mejor porque el fraude es una combinación (monto alto y noche) que los árboles captan mejor que un modelo lineal.

Modelos no supervisados y Apriori

Técnica	Resultado
K-Means	La silueta más alta es k = 2 (0.297) pero solo separa día y noche. Elegí k = 5 porque coincide con el codo y es un máximo local (0.275). Un cluster de 116 transacciones con montos muy altos tiene 31.9% de fraude, casi 15 veces el promedio.
DBSCAN	Marca 149 anomalías y el 26.85% son fraude (12 veces el promedio), pero solo captura 40 de 109 fraudes. Sirve como alerta extra.
PCA	Con 2 componentes se explica el 48.6% de la varianza y con 3 el 68.7%. Lo use para poder visualizar los clusters.
Apriori	{monto alto, noche} → fraude: soporte 0.016, confianza 0.241 y lift 11.05. Solo monto alto tiene lift 3.8, la combinación es lo que delata el fraude.

Umbral de decisión del Random Forest

Umbral	Fraudes detectados	Clientes legítimos molestados
0.1	18 de 22	15
0.2	18 de 22	3
0.3	18 de 22	1
0.4	17 de 22	0
0.5	17 de 22	0

4. Visualizaciones clave

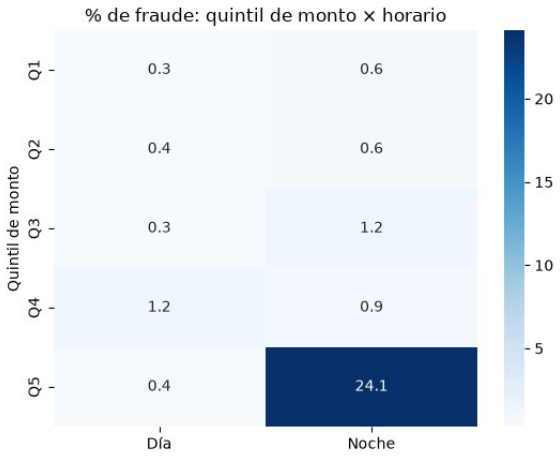


Figura 1. % de fraude por quintil de monto y horario. El fraude se concentra en monto alto de noche.

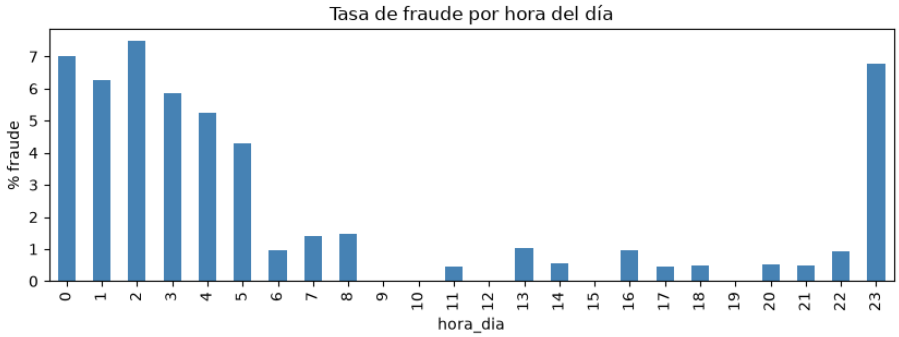


Figura 2. Tasa de fraude por hora del día. Las horas de la noche tienen mucho más fraude.

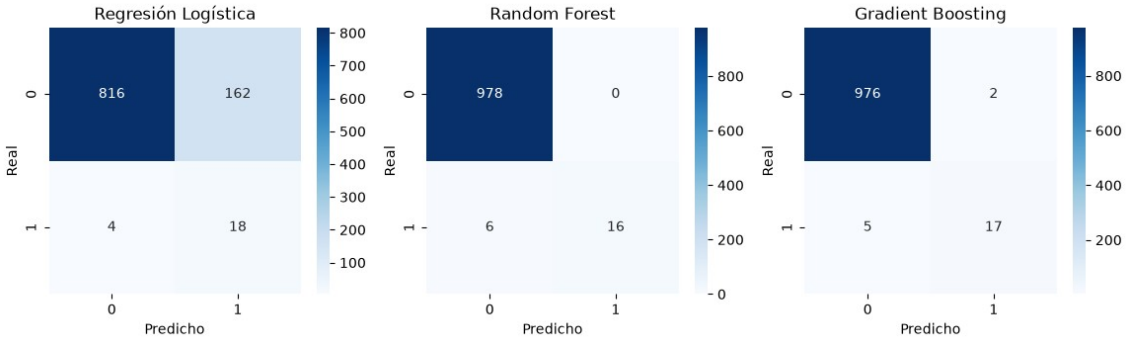


Figura 3. Matrices de confusión de los 3 modelos en el conjunto de prueba.

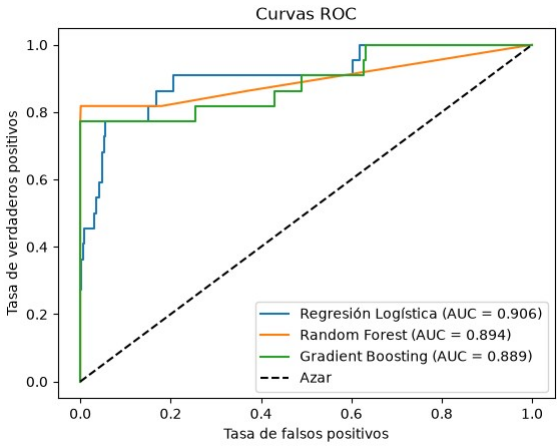


Figura 4. Curvas ROC de los 3 modelos con su AUC.

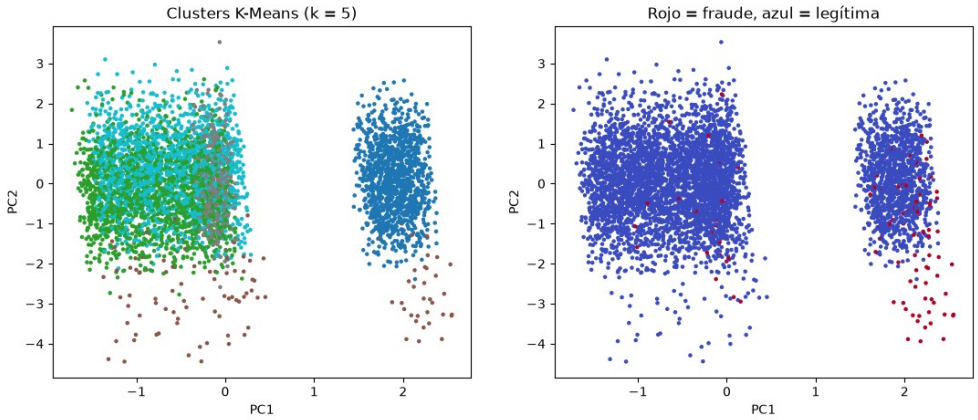


Figura 5. Clusters de K-Means en el espacio PCA (izquierda) y el mismo espacio coloreado por fraude (derecha).

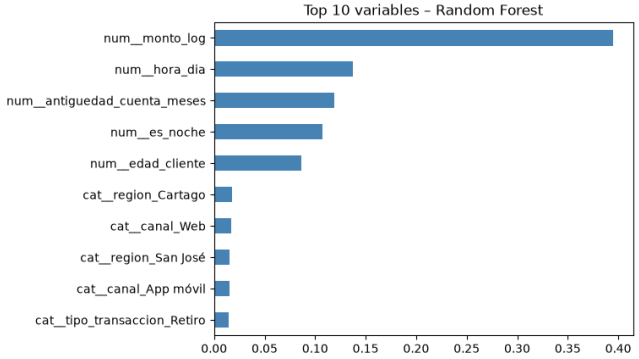


Figura 6. Top 10 variables más importantes del Random Forest.

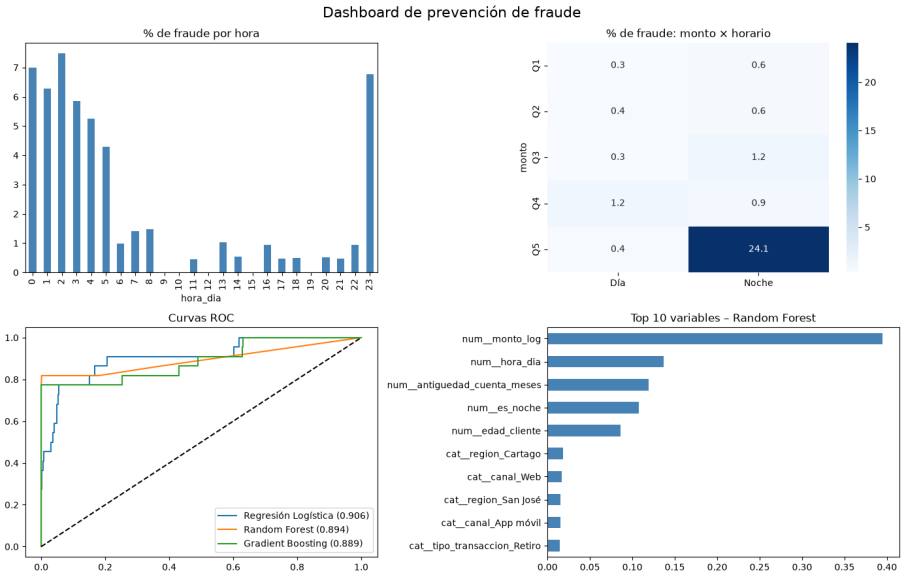


Figura 7. Dashboard con 4 gráficos: fraude por hora, monto × horario, curvas ROC e importancia de variables.

5. Interpretación y recomendaciones

¿Qué patrones distinguen el fraude? El fraude depende de cuánto y cuándo, no de quién ni por dónde. En el Random Forest monto_log (0.395), hora_dia (0.137) y es_noche (0.107) son de las variables más importantes, mientras que canal, región y tipo de transacción aportan menos de 0.02 cada una.

Cómo usarlo sin molestar al cliente? Recomiendo:

- Usar el Random Forest con **3 niveles de riesgo**: probabilidad menor a 0.3 se aprueba sin molestar, entre 0.3 y 0.5 se pide un código OTP, y 0.5 o más se bloquea y se llama al cliente.
- Mientras se implementa el modelo, aplicaria una regla simple: pedir verificación extra para montos altos entre las 22:00 y las 06:00.
- Usar DBSCAN cada mes como una alerta extra para detectar patrones nuevos de fraude.
- Revisar cada mes cuántos clientes legítimos se molestan y volver a entrenar el modelo con los fraudes confirmados.

6. Limitaciones y próximos pasos

- Los datos son sintéticos, en la vida real los patrones no son tan claros.
- Solo hay 22 fraudes en el test, entonces cada fraude que se escapa cambia el recall aprox. 4.5 puntos.
- 4 de los 22 fraudes no siguen ningún patrón visible, habria que buscar el historial directo de los clientes para comparar data.
- Los resultados del Random Forest pueden cambiar un poco según la versión de scikit-learn.

Próximos pasos: consideramos crear variables con el historial de cada cliente (cuánto gasta normalmente y cada cuánto hace transacciones), usar validación cruzada para tener métricas más estables, calcular cuánto cuesta un fraude contra cuánto cuesta molestar a un cliente para elegir mejor el umbral, y hacer una prueba piloto antes de usarlo en todo el banco.

Referencia: Fayyad, U., Piatetsky-Shapiro, G., & Smyth, P. (1996). From data mining to knowledge discovery in databases. *AI Magazine*, *17*(3), 37–54.